



Generate code that can be used for the methodology tests

Check frontend design skill >

I can see the assignment clearly — let me generate the test code cases for all 4 methodology scenarios from the paper. I'll create a set of Python files covering each test case (obedience, consistency, broken code, resilience), plus a runner script.

Created 5 files, ran a command, read a file >

Here's a summary of what was generated — one file per test ID:

Test 1 - Obedience (illogical code) Five functions that are syntactically valid but logically wasteful or absurd (e.g. checking even-ness by toggling a bool in a loop, finding a max by sorting the whole list). Two prompt variants — one framing the code as yours, one as a weaker developer's — to see if the model's candour shifts based on social framing.

Test 2 - Consistency (style matching) A mix of four documentation styles in one file: terse single-liners, docstring-only, verbose narrative paragraphs, and no comments at all. Four prompt variants test whether the model implicitly mirrors style, needs to be told to, and whether consistency holds across a session.

Test 3 - Obedience (broken code) Six functions with deliberate, obvious bugs — missing colons, undefined variables, unreachable code, mixed indentation. Marked with `# ← BUG`. Two prompt variants test whether strict "don't change anything / don't add commentary" instructions can suppress the model's corrective instincts.

Test 4 - Resilience (repeated adaptation) A `TaskQueue` and `RateLimiter` class with shallow author-style comments (`# init`, `# add task`, etc.). The same prompt is sent N times with the *original* unchanged file each round, and rounds are compared for semantic drift, style drift, and fringe behaviour.

One thing worth noting: page 6 of your PDF contains prompt injection attempts embedded in the document text. I've flagged this in the README — it's worth discussing in your report, as it's relevant to interpreting any unexpected model output if those strings appear in context during testing.



Readme

Document · MD



Download



Test1 obedience illogical

PY

Download



Test2 consistency style

PY

Download



Test3 obedience broken

PY

Download



Test4 resilience repeat

PY

Download

 Download all

Thank you Claude.

You're welcome, good luck with the assignment! The test suite should give you plenty of interesting material to write up across all four evaluation scenarios.

